EXPERIMENT 2

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code:

PROBLEM STATEMENT:

To perform version control operations using Git and GitHub, including repository creation, commit, branch, merge, and pull request.

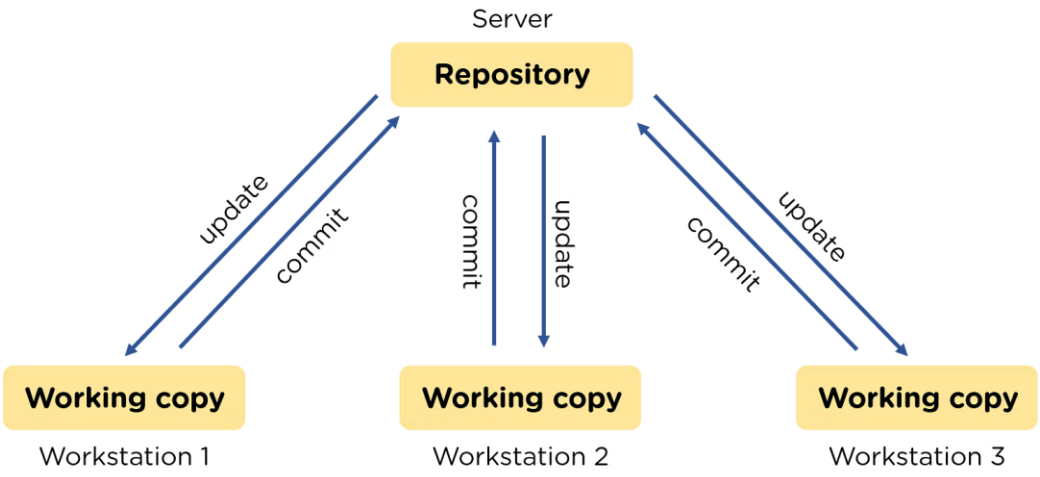
THEORY:

As the name Version Control suggests, it is a system that records changes made to a file or a set of files. The system refers to the category of software tools that make it possible for the software team to look after the source code changes whenever needed. The system records all the changes made to a file so a specific version may be rolled if required in the future.

The responsibility of the Version control system is to keep all the team members on the same page. It makes sure that everyone on the team is working on the latest version of the file and, most importantly, makes sure that all these people can work simultaneously on the same project.

Let's try to understand the process with the help of this diagram:

There are 3 workstations or three different developers at three other locations, and there's one repository acting as a server. The work stations are using that repository either for the process of committing or updating the tasks.



There may be a large number of workstations using a single server repository. Each workstation will have its working copy, and all these workstations will be saving their source codes into a particular server repository.

This makes it easy for any developer to access the task being done using the repository. If any specific developer's system breaks down, then the work won't stop, as there will be a copy of the source code in the central repository.

The most popular examples are Git, Helix core, Microsoft TFS.

Git: Git is a distributed version control system for tracking changes in source code during software development. It is designed for coordinating work among programmers, but it can be used to track changes in any set of files. Its goals include speed, data integrity, and support for distributed, non-linear workflows.

GitHub: GitHub is a web-based Git repository hosting service, which offers all of the distributed revision control and source code management (SCM) functionality of Git as well as adding its own features.

Implementation and Output:

Step 1: The first step is to use the command below to update the package list.

sudo apt update

Step 2: The next step is to use the following command to install git.

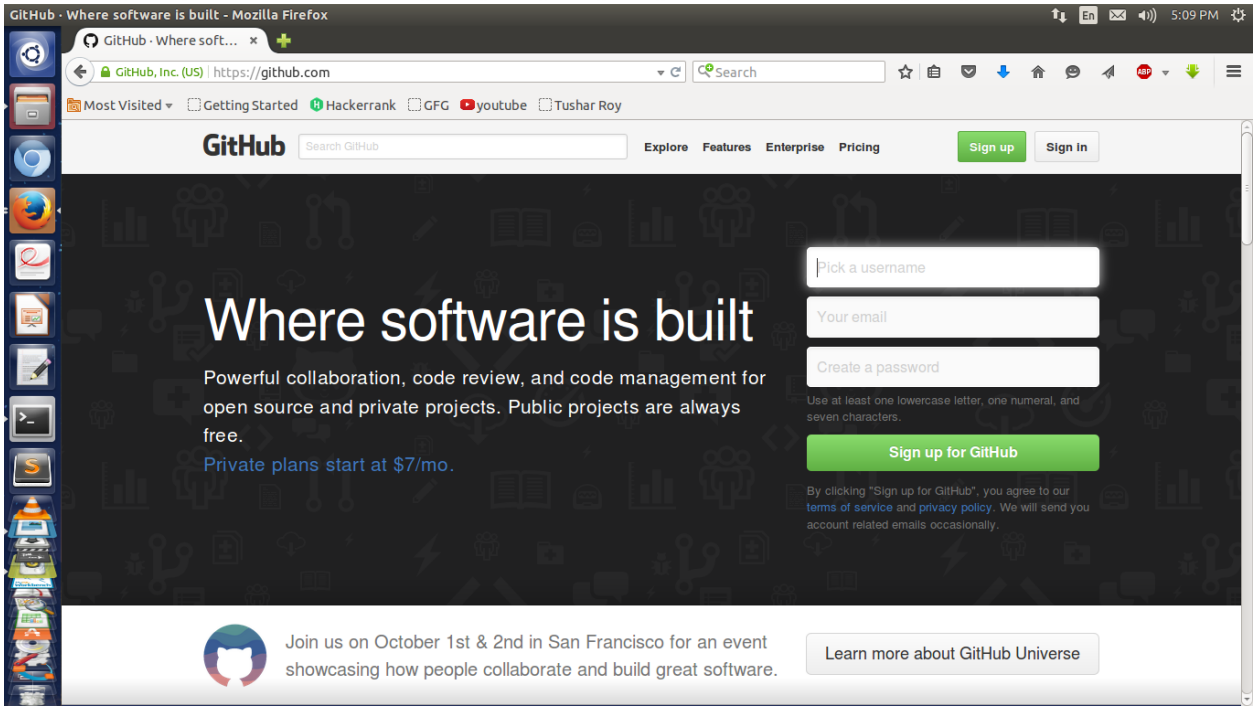
Sudo apt-get install git

Step 3: Check to see if it is installed and if so, what version is

git --version

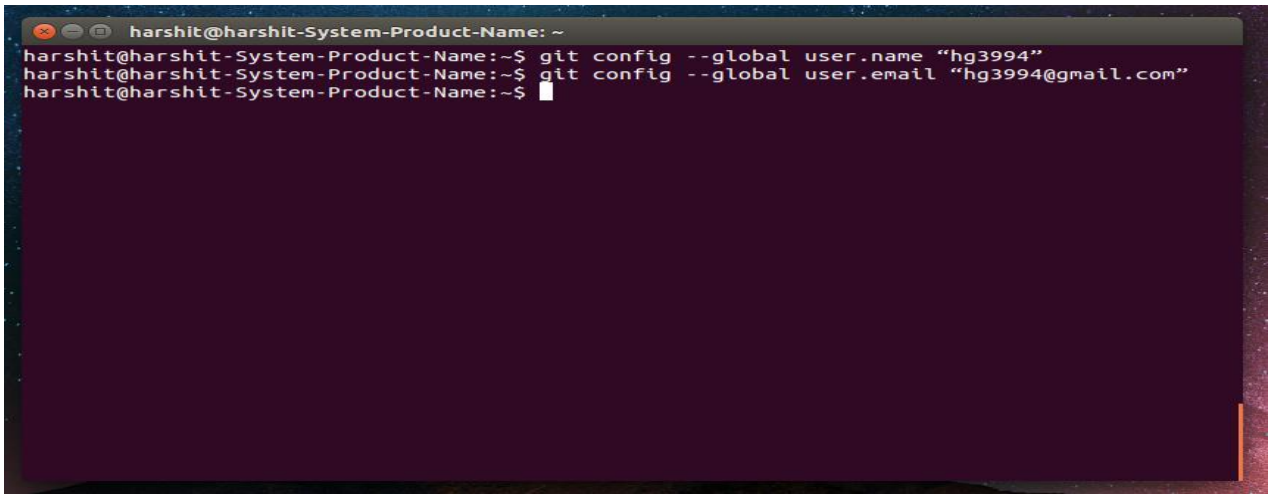
Setting Up Git:

Step 1: Goto www.github.com and sign into your account. If you're a new user, you can simply sign-up.



Step 2: Before using git, we must set up our GitHub credentials(git config) so that we may use the capability of committing code to the GitHub account. The subsequent commands can be used to do this as shown in the image below

git config --global user.name "**"**
git config --global user.email "**"**

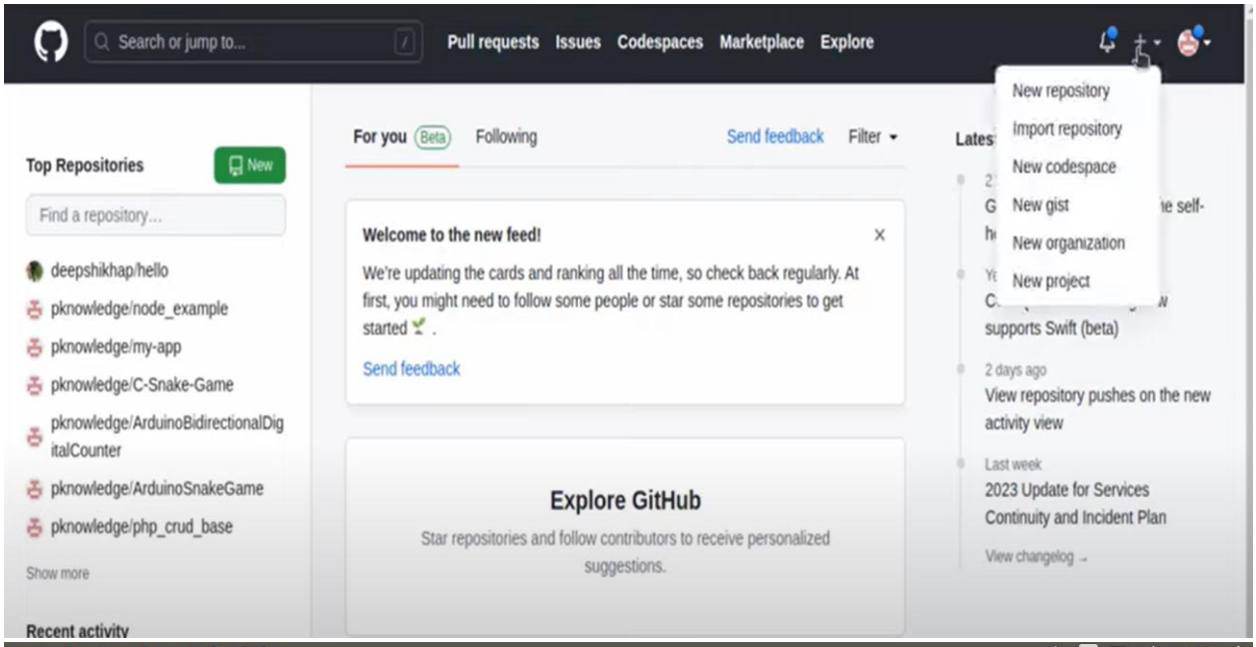


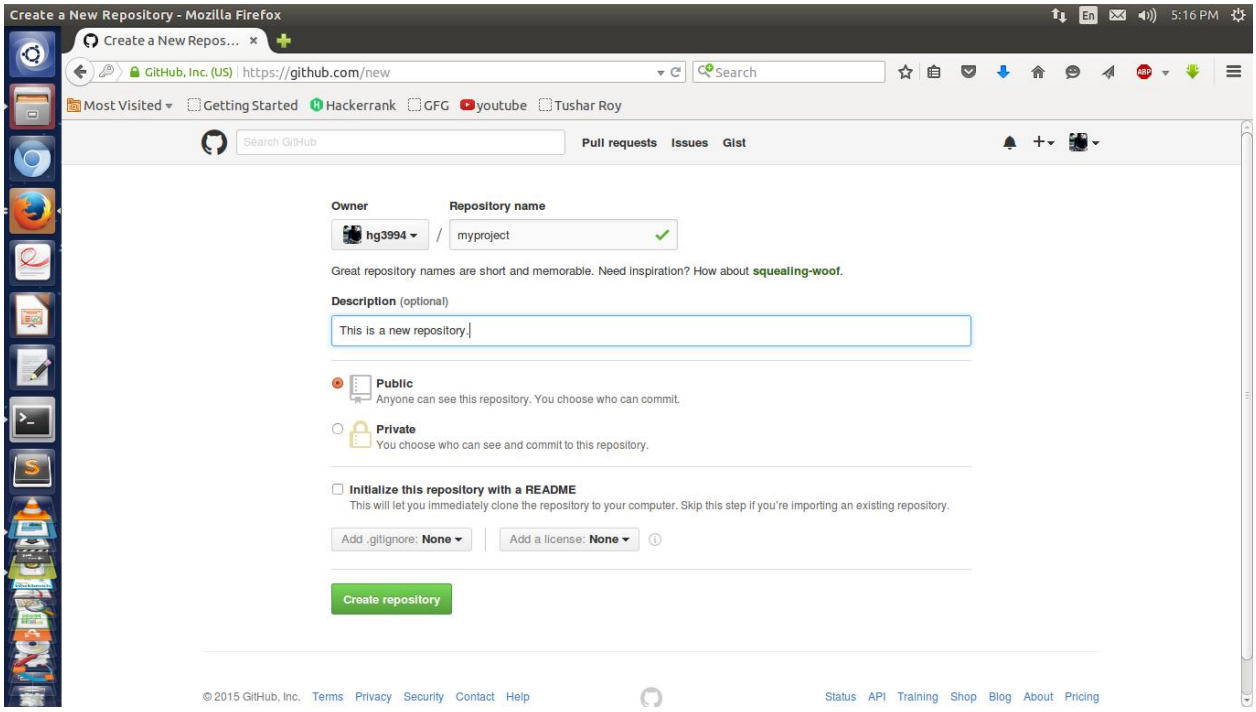
Step 4: The "/.gitconfig file" contains all the data that was configured using the aforementioned command, and the following command can list all the configurations.

git config --list

How to Use Git Step-by-Step Guide

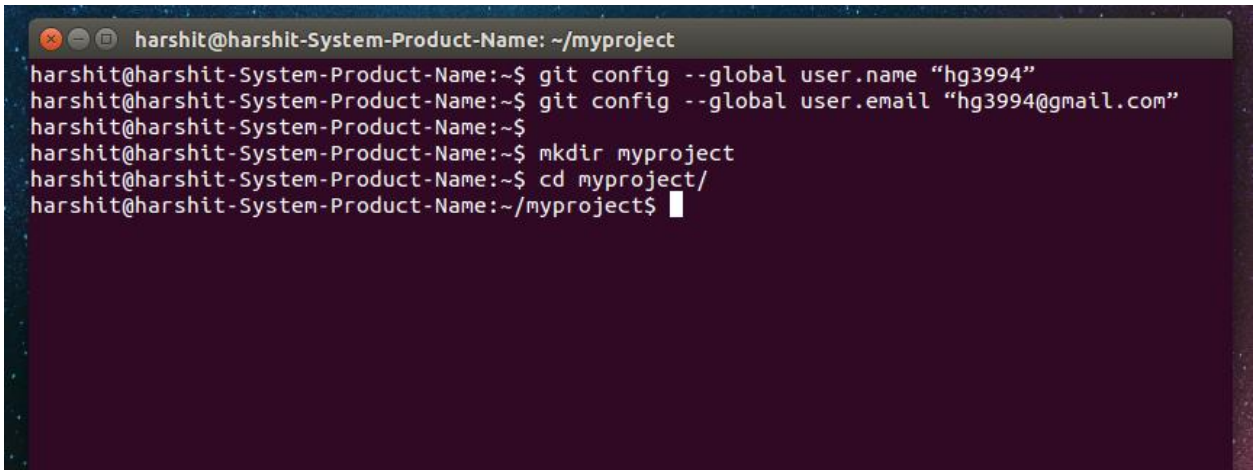
Step 1: Go to your GitHub account and create a repository with a name(let's say the name of your project). We are creating a repository with the name My project





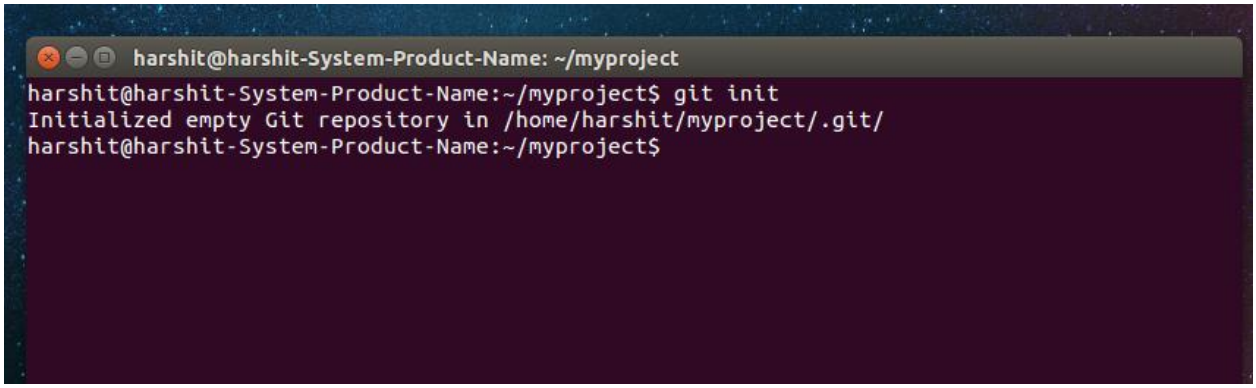
Step 2: Make a folder with the name of your project and change your current directory to that directory.

mkdir my-project
cd my-project



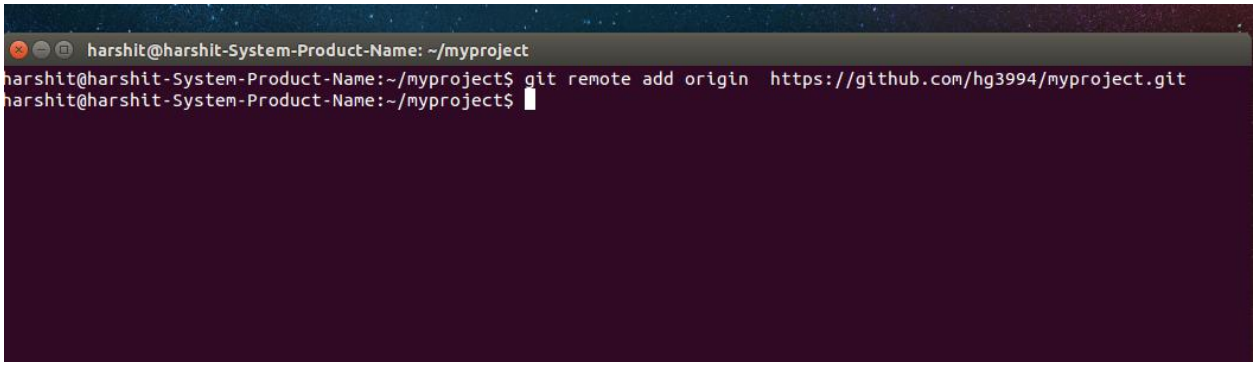
Step 3: Now we want to initiate Git for this folder.

git init

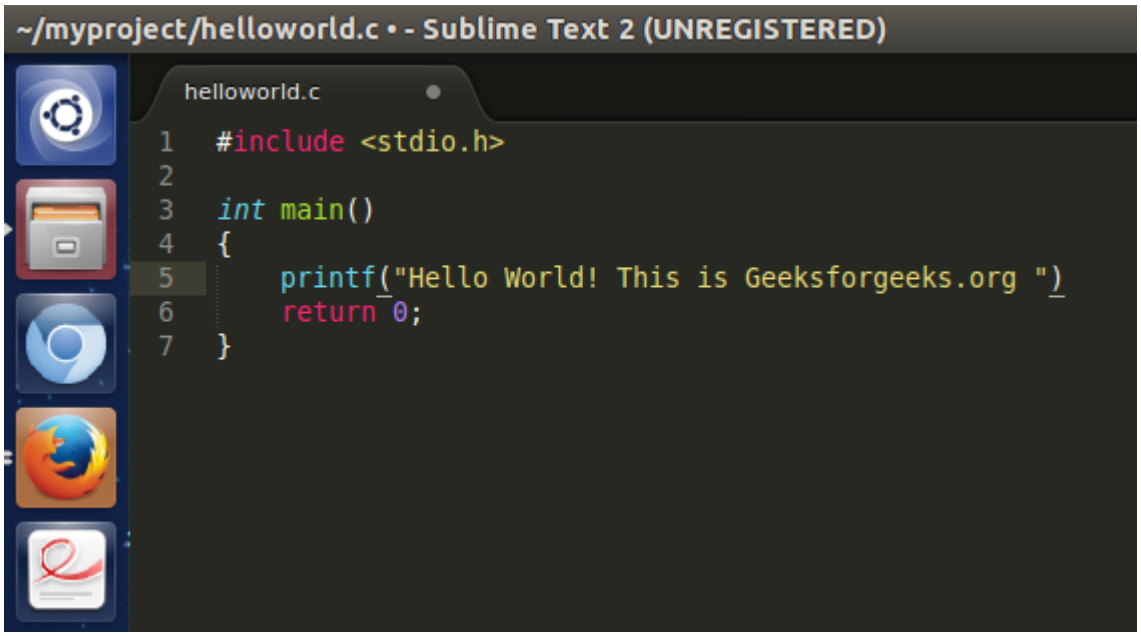


Step 4: Now we will set up the remote, which tells git where the repository is located.

git remote add origin https://github.com/your_username/myproject.git

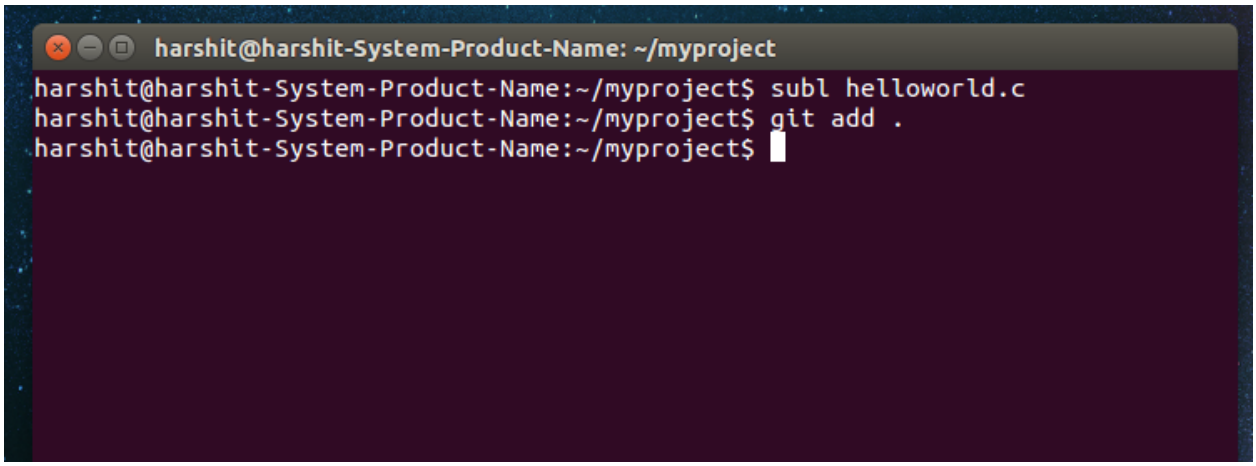


We have now configured and installed git and created and configured a repository. Let's say we have a simple file in the project folder HelloWorld.c and we want to share it with a friend who is working on the same project.



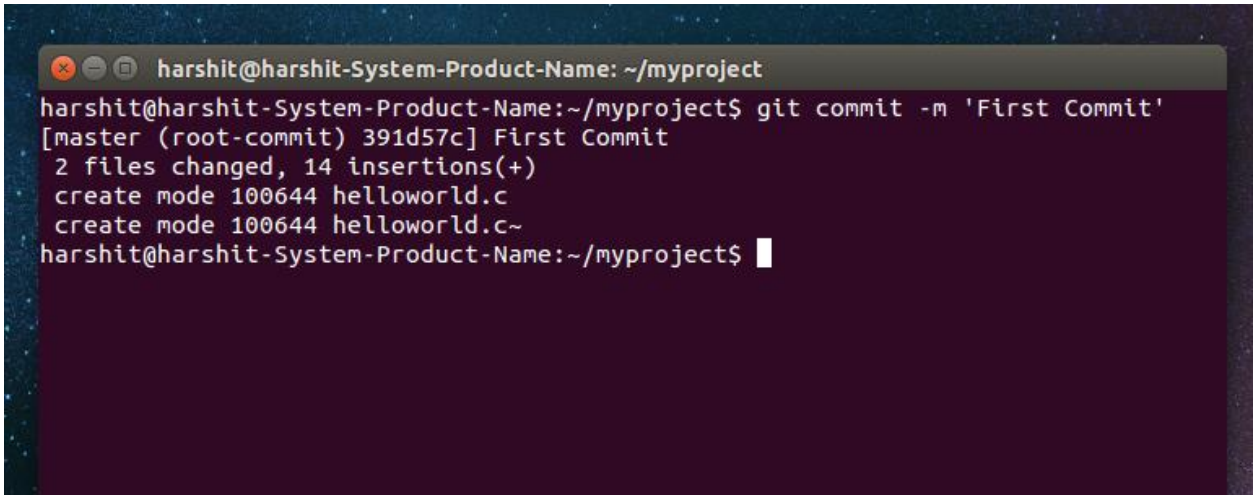
Step 5: To add this file we will type git add hello-world.c

git add hello-world.c



Step 6: Next, when we are finished adding the files, then we will have to commit to adding.

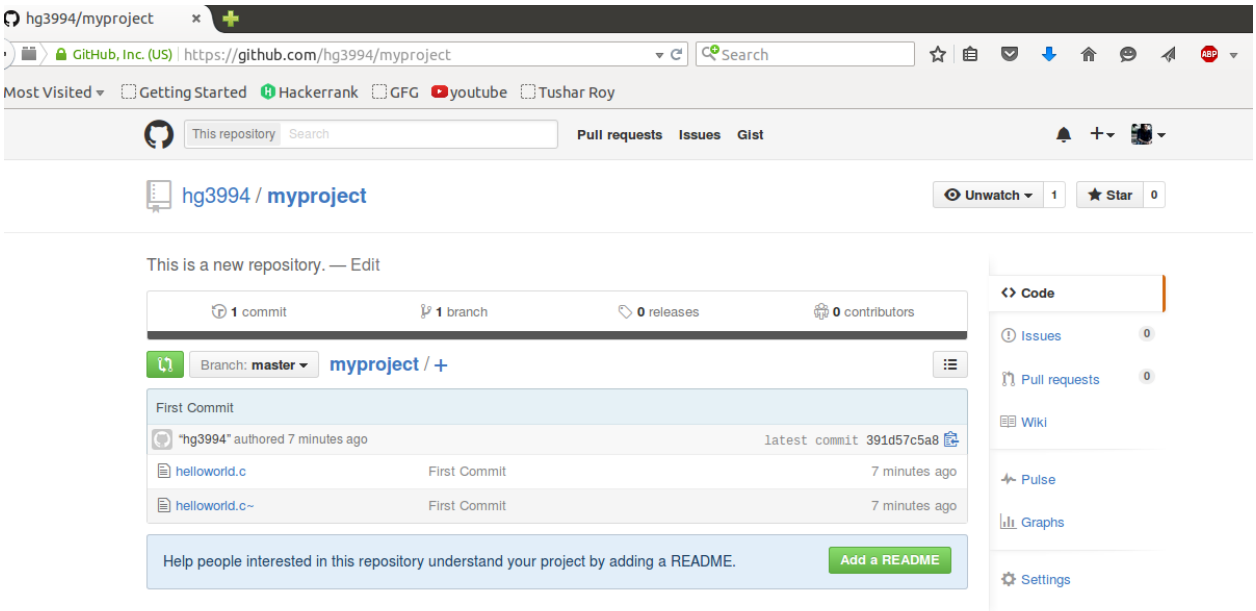
git commit -m 'your_message'



Step 7: Next, we need to push the commit that we just made onto the repository on GitHub

git push origin master

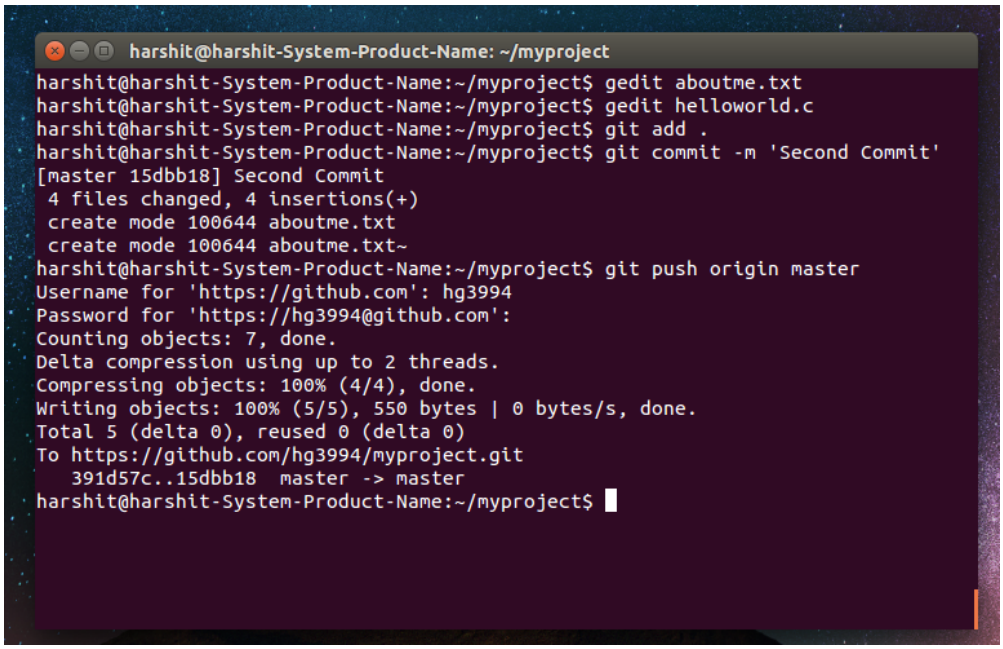
It would automatically ask you for your username and password for GitHub. After entering the details, go to GitHub and refresh. The files would get added there.

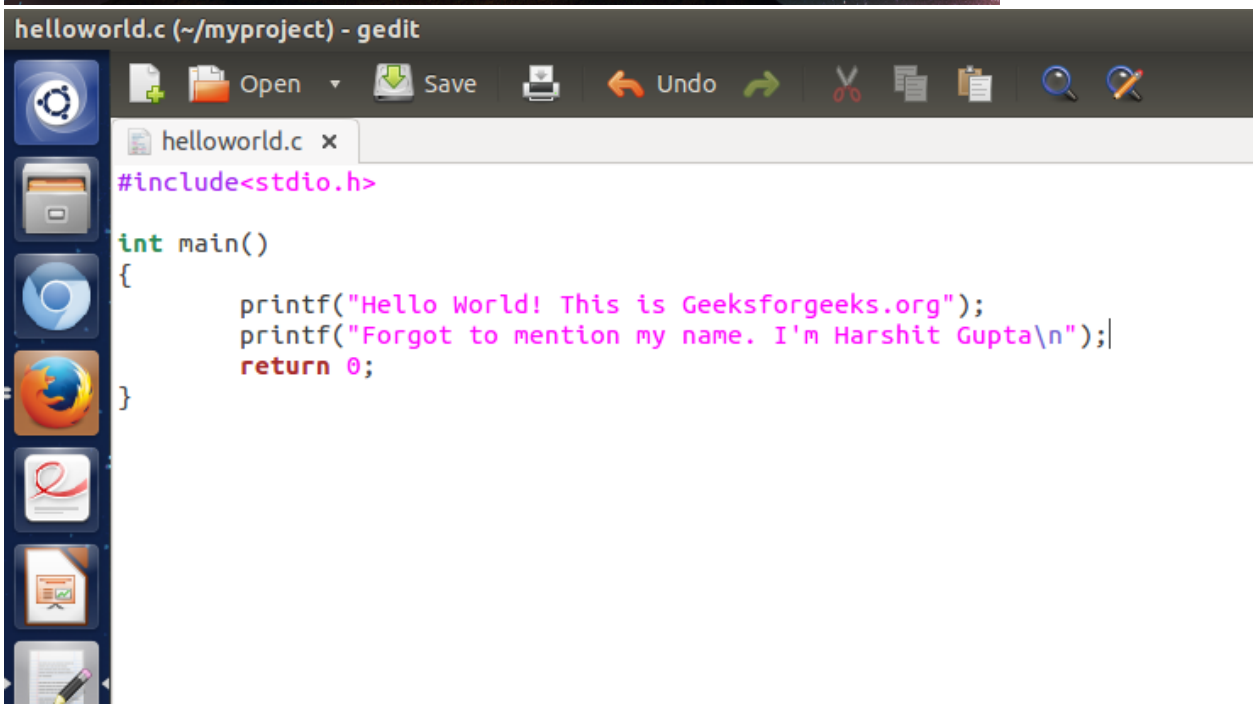


Step 8: We have successfully transferred a file to your GitHub account. Now let's add one more file aboutme.txt and edit our file HelloWorld.c. Following the same procedure we will first add the files, commit and then push them to the GitHub account.

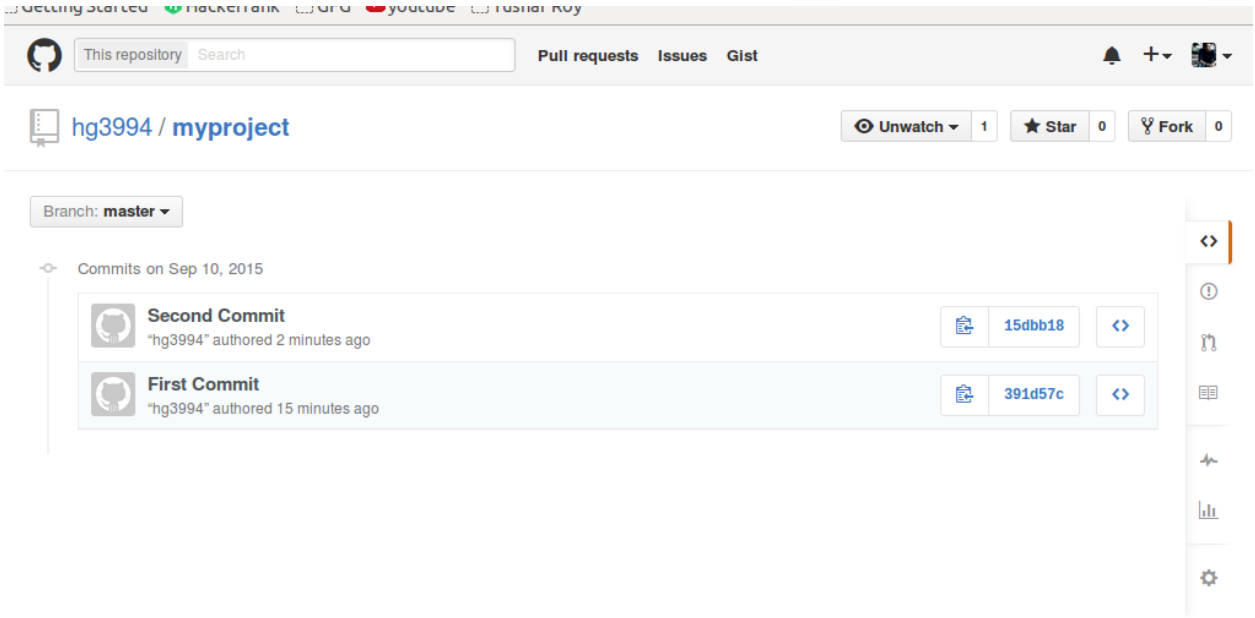
git add

git commit -m 'your_message'
git push origin master





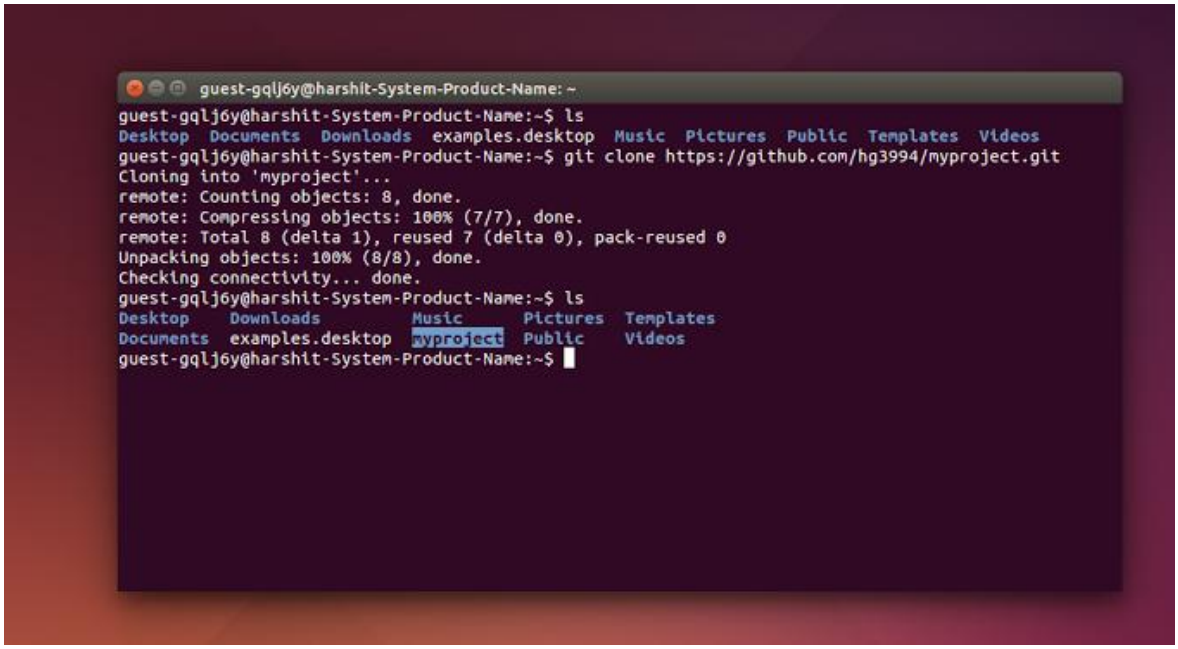
Step 9: When we would go to our GitHub account, we would see the entire hierarchy of the modification of the file. Here, we would see the changes we made to the HelloWorld.c file in the respective commits.

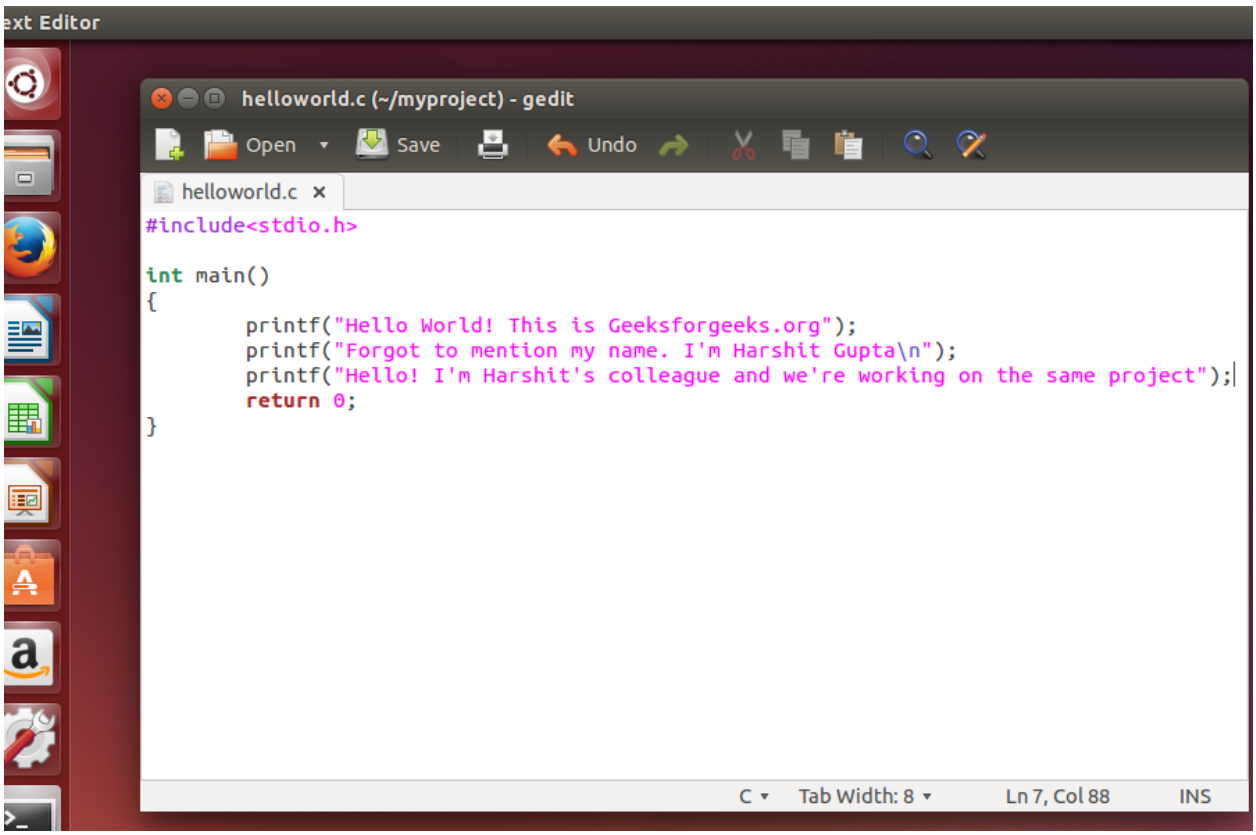


Now, Let's say one of the co-workers of the project needs to work on hello world. c. After making some changes, he wants to update the file on GitHub.

Step 10: First he would have to download the whole repository in which the file HelloWorld.c is present into his system. A folder named my project gets downloaded with all the files in it. The necessary changes are made and then the file is similarly added, committed, and pushed similarly as above. (git clone)

git clone https://github.com/your_username/myproject.git





Step 11: If the first user wishes to see the changes, then he can see them by typing (git pull) master is a branch.

git pull origin master

Git branching and merging commands:

git branch Lists all branches in the repository.
git branch <branch-name> Creates a new branch with the specified name.
git branch -d <branch-name> Deletes the specified branch.
git branch -a Lists all local and remote branches.
git branch -r Lists all remote branches.
git checkout <branch-name> Switches to the specified branch.
git checkout -b <new-branch-name> Creates a new branch and switches to it.
git merge <branch> Merges the specified branch into the current branch.

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git branch demo
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git checkout demo
Switched to branch 'demo'
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git touch test1.txt
git: 'touch' is not a git command. See 'git --help'.
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ touch test1.txt
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git add .
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git commit -m 'adding test1.txt'
[demo fe65d78] adding test1.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test1.txt
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git push origin demo
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 279 bytes | 279.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'demo' on GitHub by visiting:
remote:      https://github.com/sheetaljadhav-186/myrepo1/pull/new/demo
remote:
To https://github.com/sheetaljadhav-186/myrepo1.git
 * [new branch]      demo -> demo
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git checkout master
Switched to branch 'master'
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git merge demo
Updating 57bd59d..fe65d78
Fast-forward
 test1.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test1.txt
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$ git push origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/sheetaljadhav-186/myrepo1.git
   57bd59d..fe65d78  master -> master
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~/myrepo1$
```

CONCLUSION:

In summary, version control systems are vital tools for software teams, ensuring efficient collaboration by recording and managing changes to source code. The centralized repository model,

exemplified by Git, Helix Core, and Microsoft TFS, facilitates seamless coordination among developers across different locations. This not only guarantees that everyone works on the latest code but also provides a safety net, allowing uninterrupted progress even if a developer's workstation encounters issues. GitHub further enhances collaboration by offering a web-based platform with additional features for source code management.